

W02 报告

1. 测试范围与环境

1.1 测试范围

- 默认单测：`dart test --exclude-tags integration`
- 集成单测：`dart test --tags integration`
- `datasets/samples` 三文件探针，coco 50 张与 rvlcdip 16 张入库探针
- `lib/` 行覆盖率，`--report-on=lib`
- `apps/desktop` 的 flutter analyze

1.2 测试环境

操作系统：Microsoft Windows 11 25H2 Dart: 3.13.4 (stable) Flutter: stable 3.47.5, 安装路径 D:\flutter Java: 17.0.4 LTS, JAVA_HOME 为 C:\Program Files\Java\jdk-17.0.4 Apache Tika: tika-app 4.0.0, tools/tika-app 解析包: packages/core_parsing

2. 用例统计

模块	用例 ID	用例数	通过	失败	未测
TXT	PAR-01 ~ PAR-07	7	7	0	0
注册表	PAR-08 ~ PAR-10	3	3	0	0
DOCX	PAR-11 ~ PAR-14	4	4	0	0
Tika CLI	PAR-15 ~ PAR-18	4	4	0	0
PDF	PAR-19 ~ PAR-21	3	3	0	0
图像	PAR-22 ~ PAR-24	3	3	0	0
扫描	ING-01 ~ ING-03	3	3	0	0
元数据	ING-04 ~ ING-06	3	3	0	0
入库	ING-07 ~ ING-10	4	4	0	0
合计	—	34	34	0	0

覆盖率单独见 覆盖率报告.md，不计入上表。

默认套件结束行是 00:01 +98: All tests passed!。集成套件结束行是 00:06 +5: All tests passed!。--reporter expanded 的进度刷新被采集成重复行，同一条用例名会出现多次。判定看结束行，不看重复次数。

3. 用例表

命令在 packages/core_parsing 下执行，除非另注「仓库根目录」。

用例 ID	命令	期望	实际	判定	证据
PAR-01	dart test --exclude-tags integration	UTF-8 正文与夹具一致	utf-8 fixture keeps Chinese and normalizes CRLF: 正文 Hello core_parsing. 加换行和 第二行: 中文内容。， outcome 为 ok	通过	logs/dart_test.txt
PAR-02	同上	UTF-8 BOM 正文一致，标记被去掉	utf-8 BOM fixture strips the mark: 正文 BOM prefixed ASCII and 中文。， 无 warning	通过	logs/dart_test.txt
PAR-03	同上	UTF-16 中文一致	LE 夹具正文 UTF-16LE 中文；BE 字节正文 BE 中文。两条都是 ok	通过	logs/dart_test.txt
PAR-04	同上	GBK 中文一致	gbk fixture falls back and records the warning: 正文 GBK: 中文测试， warning 为 encoding-fallback:gbk	通过	logs/dart_test.txt
PAR-05	同上	空文件为 ok	empty file is ok with an empty body: 正文空，一页，ok	通过	logs/dart_test.txt
PAR-06	同上	无换行的大单行，未超上限，为 ok	a newline-free line under the ceiling stays ok: 一行 520 字符，无换行，走默认上限。正文与原文一致，warnings 为空，outcome 为 ok	通过	logs/dart_test.txt
PAR-07	同上	超过上限为 partial，并带截断标记	bytes past the ceiling are partial and truncated: TxtParser(maxBytes: 4) 读 abcdefg，正文 abcd，warning truncated:4，partial	通过	logs/dart_test.txt
PAR-08	同上	已知扩展名命中解析器	a matching parser is invoked: A.TXT 调用 stub	通过	logs/dart_test.txt
PAR-09	同上	未知扩展名为 unsupportedFormat，不抛异常	an unknown extension is skipped and does not throw: picture.bin 的 kind 为 unsupportedFormat，outcome 为 skipped	通过	logs/dart_test.txt
PAR-10	同上	.TXT / .Pdf 命中	extensions match regardless of case or a leading dot: .TXT、Pdf、Notes.Pdf 都命中 stub	通过	logs/dart_test.txt
PAR-11	同上	正常 docx 正文非空，并带 title / creator	minimal fixture yields paragraphs and core properties: 夹具 title 为 Core Parsing Fixture。sample.docx has body text and a creator: 正	通过	logs/dart_test.txt

用例 ID	命令	期望	实际	判定	证据
			文含 Apache Tika, creator 为 Mengxiao Wu。dc:title 是空元素, 测试只断言这个键存在		
PAR-12	同上	缺少 word/document.xml 为 failed	a zip without document.xml is failed: corruptedInput, 消息含 word/document.xml	通过	logs/dart_test.txt
PAR-13	同上	损坏 zip 为 failed	a corrupt zip is failed and names the reason: corruptedInput, 消息含 zip	通过	logs/dart_test.txt
PAR-14	同上	没有 docProps 时正文可用, title 为 null	tabs and line breaks inside a paragraph are kept: 正文 A、制表符、B、换行、C。夹具只有 word/document.xml。测试没有断言 title。实现上缺少 core.xml 时属性表为空	通过	logs/dart_test.txt
PAR-15	同上	-x 多页成功	xhtml pages are split and entities are decoded: 假 ProcessRunner, 按 page div 切页。不启动 Java	通过	logs/dart_test.txt
PAR-16	同上	没有 page div 时回退 -t, 得到单页	xhtml without page divs falls back to plain text: 第二次调用带 -t, 正文为 fallback。-x 退出码非 0 且没有页时也会回退, 见 a failed xhtml run falls back to plain text。超时不回退	通过	logs/dart_test.txt
PAR-17	同上	进程非 0 退出为失败, stderr 保留	both modes failing throws and keeps stderr off the message: 两次退出码都是 2, 抛 TextExtractionException, stderr 为 java failed, 消息不含 stdout。PdfParser 把该异常记成 corruptedInput / failed	通过	logs/dart_test.txt
PAR-18	同上	Java 缺失时降级, 不崩溃	提取器测试抛 ProcessStartException。a missing java runtime is dependencyUnavailable: PdfParser 记 dependencyUnavailable / failed。任务表里的名字是 ParserUnavailable, 代码里的 kind 是 dependencyUnavailable	通过	logs/dart_test.txt
PAR-19	dart test --tags integration	sample.pdf 为 2 页、正文非空、ok	sample.pdf yields two pages through Tika: pageCount 为 2, 正文含 Apache Tika。同一次集成套件里还有 Tika CLI 的 sample.pdf	通过	logs/dart_test.txt; logs/ingest_samples.txt

用例 ID	命令	期望	实际	判定	证据
			yields two pages through the real Tika CLI。探针: sample.pdf outcome=ok ms=5215 pages=2 chars=4358		
PAR-20	dart test --exclude-tags integration	加密 PDF 为 failed, 并记下原因	an encrypted pdf keeps the extractor reason: 假提取器抛出 PDF is encrypted., kind 为 corruptedInput, detail 为 stderr	通过	logs/dart_test.txt
PAR-21	同上	0 字节 PDF 为 failed	an empty pdf fails before the extractor runs: 消息 PDF is empty., corruptedInput, 提取器未被调用	通过	logs/dart_test.txt
PAR-22	同上	PNG 给出尺寸和格式	sample.png reports size, format, and rgb: 1186×1137, format 为小写 png, color 为小写 rgb。探针同一尺寸, ms=80	通过	logs/dart_test.txt; logs/ingest_samples.txt
PAR-23	同上	JPEG 给出尺寸和格式	a jpeg reports format and a positive size: 包内夹具 tiny.jpg。COCO 的 000000006818.jpg 在集成套件, 不在默认套件	通过	logs/dart_test.txt
PAR-24	同上	伪装成图片的非图片为 failed	a non-image with an image extension fails。另有 an empty image file fails as corrupted input: 空字节 PNG 为 corruptedInput	通过	logs/dart_test.txt
ING-01	同上	递归结果包含子目录文件	fixtures include nested files and skip hidden and temp names: 结果含 nested/a.txt, 且已按路径排序	通过	logs/dart_test.txt
ING-02	同上	不含 ~\$ 和点开头的名字	同一条夹具测试要求点开头与 ~\$ 为空。hidden directories, temp files, and system directories are skipped: 目录里只剩下 keep.txt	通过	logs/dart_test.txt
ING-03	同上	只保留白名单扩展名	上一条临时目录里的 notes.bin 不在结果中。白名单是 .txt .pdf .docx .jpg .jpeg .png	通过	logs/dart_test.txt
ING-04	同上	同一文件两次 SHA-256 一致	content hash is stable and times are present: 两次 contentHash 相等	通过	logs/dart_test.txt
ING-05	同上	时间字段非空	同一条测试: modifiedAt、accessedAt、createdAt 都非空。createdAt 来自 stat.changed	通过	logs/dart_test.txt

用例 ID	命令	期望	实际	判定	证据
ING-06	同上	中文路径不抛异常	同一条测试的相对路径是 中文目录/说明.txt, 文件名 说明.txt	通过	logs/dart_test.txt
ING-07	dart test --tags integration; 仓库根目录再跑两条 ingest_probe	50 + 16 = 66 条, 0 崩溃	集成测试 coco and rvlcdip yield 66 image records without failures。探针: COCO total: 50 ok: 50 failed: 0 ms: 1400; rvlcdip total: 16 ok: 16 failed: 0 ms: 651	通过	logs/dart_test.txt; logs/ingest_samples.txt
ING-08	dart test --exclude-tags integration	坏文件被隔离, 其余文件仍成功	a corrupt file is isolated and the rest still succeed: total 2, succeeded 1, failed 1, cancelled 为 false	通过	logs/dart_test.txt
ING-09	同上	空目录得到空报告	an empty directory is an empty report: total 0, cancelled 为 false	通过	logs/dart_test.txt
ING-10	同上	取消后停止, 报告 cancelled	cancel before start yields a cancelled empty report: cancelled, total 0。cancel after the first file stops the rest: cancelled, total 小于 4 且大于 0	通过	logs/dart_test.txt

4. 探针与静态检查

这些不在矩阵编号里, 对应验收清单第 1、3、4、5 条。

检查	命令	实际	证据
包静态检查	packages\core_parsing 下 dart analyze	No issues found!	logs/dart_analyze.txt; screenshots/dart-analyze.png
默认测试	dart test --exclude-tags integration	00:01 +98: All tests passed!	logs/dart_test.txt; screenshots/dart-test.png
集成测试	dart test --tags integration	00:06 +5: All tests passed!	同上
samples 探针	仓库根目录 dart run packages\core_parsing\bin\parse_probe.dart --path datasets\samples	docx ok, 80 ms, 1 页, 1499 字符; pdf ok, 5215 ms, 2 页, 4358 字符; png ok, 80 ms, 1186×1137, png / rgb。summary files=3 ok=3 failed=0	logs/ingest_samples.txt; screenshots/parse-probe.png
COCO	dart run packages\core_parsing\bin\ingest_probe.dart	total 50, ok 50, failed 0, 1400 ms	logs/ingest_samples.txt; screenshots/ingest-coco.png

检查	命令	实际	证据
	--path datasets\coco\val_sample\images		
rvlcdip	dart run packages\core_parsing\bin\ingest_probe.dart --path datasets\rvlcdip\sample	total 16, ok 16, failed 0, 651 ms	logs\ingest_samples.txt; screenshots\ingest- rvlcdip.png
桌面工程	apps\desktop 下 flutter analyze	No issues found! (ran in 7.6s)	logs\flutter_analyze.txt; screenshots\flutter- analyze.png
覆盖率	见 覆盖率报告.md	949 / 1043 = 90.99%	logs\coverage.txt; coverage\lcov.info; screenshots\coverage- percent.png

ingest_samples.txt 里，docx 和 pdf 的预览行在采集时把省略号和下一行拼到了一起。上表用的是 outcome、页数、字符数和 summary，不依赖那两行预览的换行。

samples 探针里只有 pdf 启动 JVM。Day 3 同目录的 pdf 是 4857 ms (logs/day3_jvm_single_file.txt) 。Day 4 这次是 5215 ms。这两次只记了总时间。

2026-10-10 对 datasets/samples/sample.pdf 又跑了 3 次单文件探针，原始输出在 reports/week3/logs/pdf_stage_timing.txt。Java 17.0.4。三次都是 ok，2 页，4358 字符，71250 字节。total_ms 依次是 5101、4858、5137，中位数 5101。

阶段	第 1 次	第 2 次	第 3 次	最小	中位	最大
stat	1	2	2	1	2	2
process_start	40	39	47	39	40	47
process_wait	5050	4804	5076	4804	5050	5076
stream_drain	0	0	0	0	0	0
xhtml_split	3	4	3	3	3	4
text_fallback	0	0	0	0	0	0
assemble	1	2	1	1	1	2
total_ms	5101	4858	5137	4858	5101	5137

stat 是读取文件信息并判断是不是空文件。process_start 是创建 Java 进程。process_wait 从进程已启动到退出，含 JVM 启动、扫描 classpath 和 Tika -x。stream_drain 是收完 stdout 和 stderr。xhtml_split 是按页切开。text_fallback 为 0，因为 -x 已经分出 2 页。assemble 是拼出解析结果。三次的各段之和分别是 5095、4851、5129，比 total_ms 少 6 到 8 ms，差在挂接流监听。process_wait 三次都约占 total_ms 的 99%。这一段还没有拆成 JVM 启动和正文抽取。

```

accessible-multimodal-retrieval\packages\core_parsing on 主 main [!+] is v0.1.0 via
● > dart test --coverage=coverage --exclude-tags integration
00:08 +97: All tests passed!

accessible-multimodal-retrieval\packages\core_parsing on 主 main [!+] is v0.1.0 via took 10s
● > dart run coverage:format_coverage --lcov --in=coverage --out=coverage/lcov.info --report-on=lib

accessible-multimodal-retrieval\packages\core_parsing on 主 main [!+] is v0.1.0 via took 3s
> $lf = 0; $lh = 0
· Get-Content coverage\lcov.info | ForEach-Object {
·   if ($_ -match '^LF:(\d+)') { $lf += [int]$Matches[1] }
·   if ($_ -match '^LH:(\d+)') { $lh += [int]$Matches[1] }
· }
· "lines hit $lh / $lf = $([math]::Round(100.0 * $lh / $lf, 2))%"
lines hit 949 / 1043 = 90.99%

```

```

accessible-multimodal-retrieval\packages\core_parsing on 主 main [+] is v0.1.0 via
● > dart analyze
Analyzing core_parsing...
No issues found!

```

```

accessible-multimodal-retrieval\packages\core_parsing on 主 main [!?] is v0.1.0 via via v3.13.2 (.venv)
● > dart test --exclude-tags integration
00:02 +98: All tests passed!

accessible-multimodal-retrieval\packages\core_parsing on 主 main [!?] is v0.1.0 via via v3.13.2 (.venv) took 4s
● > dart test --tags integration
00:06 +5: All tests passed!

```

```

accessible-multimodal-retrieval on 主 main [!+]
● > java -version
java version "17.0.4" 2022-07-19 LTS
Java(TM) SE Runtime Environment (build 17.0.4+11-LTS-179)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.4+11-LTS-179, mixed mode, sharing)

accessible-multimodal-retrieval on 主 main [!+]
● > dart run packages\core_parsing\bin\parse_probe.dart --path datasets\samples
root: E:\Project\accessible-multimodal-retrieval\datasets\samples
sample.docx outcome=ok ms=87 pages=1 chars=1499
  preview: Apache Tika Documentation Table of Contents Where to start This reference guide was generated with the
  assistance of AI ...
sample.pdf outcome=ok ms=4546 pages=2 chars=4358
  preview: Apache Tika Documentation Table of Contents • Where to start This reference guide was generated with t
  he assistance of A...
sample.png outcome=ok ms=74 pages=0 chars=0 format=png size=1186x1137 color=rgb
summary files=3 ok=3 partial=0 skipped=0 failed=0

```

```

accessible-multimodal-retrieval on 主 main [!+] took 9s
● > dart run packages\core_parsing\bin\ingest_probe.dart --path datasets\coco\val_sample\images
root: E:\Project\accessible-multimodal-retrieval\datasets\coco\val_sample\images
total: 50 ok: 50 partial: 0 skipped: 0 failed: 0 cancelled: false ms: 1251
000000006818.jpg ok 77539 B
0000000017627.jpg ok 186973 B

```

```

accessible-multimodal-retrieval on  main  took 5s
● > dart run packages\core_parsing\bin\ingest_probe.dart --path datasets\rvlcdip\sample
root: E:\Project\accessible-multimodal-retrieval\datasets\rvlcdip\sample
total: 16 ok: 16 partial: 0 skipped: 0 failed: 0 cancelled: false ms: 631
0_0.png ok 108418 B
10_0.png ok 228687 B
11_0.png ok 128866 B
12_0.png ok 43351 B
13_0.png ok 276721 B
14_0.png ok 261272 B
15_0.png ok 307372 B
1_0.png ok 273415 B
2_0.png ok 23624 B
3_0.png ok 333982 B
4_0.png ok 302293 B
5_0.png ok 765469 B
6_0.png ok 120818 B
7_0.png ok 244928 B
8_0.png ok 35499 B
9_0.png ok 282455 B

```

```

accessible-multimodal-retrieval\apps\desktop on  main  is  v1.0.0+1 via 
● > flutter analyze
Analyzing desktop...
No issues found! (ran in 7.3s)

```

5. 证据索引

路径	对应内容
reports/week2/logs/dart_analyze.txt	PAR 默认套件所在包的静态检查
reports/week2/logs/dart_test.txt	PAR-01..18、PAR-20..24、ING-01..06、ING-08..10, 以及 PAR-19、PAR-23 的 coco 条、ING-07
reports/week2/logs/coverage.txt	D3 命令和百分比
reports/week2/logs/ingest_samples.txt	PAR-19、PAR-22、ING-07 的探针, 以及 Java 版本
reports/week2/logs/flutter_analyze.txt	验收清单第 5 条
reports/week2/logs/day3_jvm_single_file.txt	Day 3 的单文件 JVM 耗时, 已在 fb5a162
reports/week3/logs/pdf_stage_timing.txt	2026-10-10 对 sample.pdf 的三次分段耗时
reports/week2/coverage/lcov.info	D3 的 lcov
reports/week2/覆盖率报告.md	百分比说明
reports/week2/screenshots/*.png	第 4 节各条命令的终端截屏
docs/api/parsing-module-api.md	D5
docs/tdd/technical-design-document.md	D6

6. 结论与限制

解析包可以在默认套件全绿的前提下解析四种文本路径和两种图像，并把 coco 50 张与 rvlcdip 16 张记成报告。坏文件、空目录、取消和坏根目录都落在报告里。覆盖率 90.99%。

限制：

- PAR-14 的测试没有断言 title 缺失。
- dart_test.txt 没有逐条列出全部 98 个名字。expanded 进度被拆行了，结束行是 +98 和 +5。
- 覆盖率不含真 Java。sample.pdf 的 2 页在集成套件和探针里，不在 lcov 里。
- PDF 单文件 5215 ms，含一次 JVM 启动。W2 仍是每个 PDF 一个进程。2026-10-10 的三次分段里，时间主要在 process_wait（中位 5050 ms，总时间中位 5101 ms）。这一段仍同时包含 JVM 启动和 Tika 抽取。
- 图像格式字符串是小写 png / jpeg，色彩模式是小写 rgb。
- DOCX 解析器不调用 Tika。架构图里的 DOCX 回退没有实现。
- 数据集图片不在 git 里。干净克隆跑不了 dart test --tags integration 里的 coco / rvlcdip 两条，除非本机自备这些文件。

系统架构设计文档

1. 目标与约束

1.1 产品目标

做一个**完全离线**、开源、可在 Windows / macOS / Linux 运行的桌面工具：把本机 PDF、文档、图片建成本地索引，用自然语言或关键词搜回来，并保证视障用户能独立完成「搜索 → 看结果 → 打开」全流程。

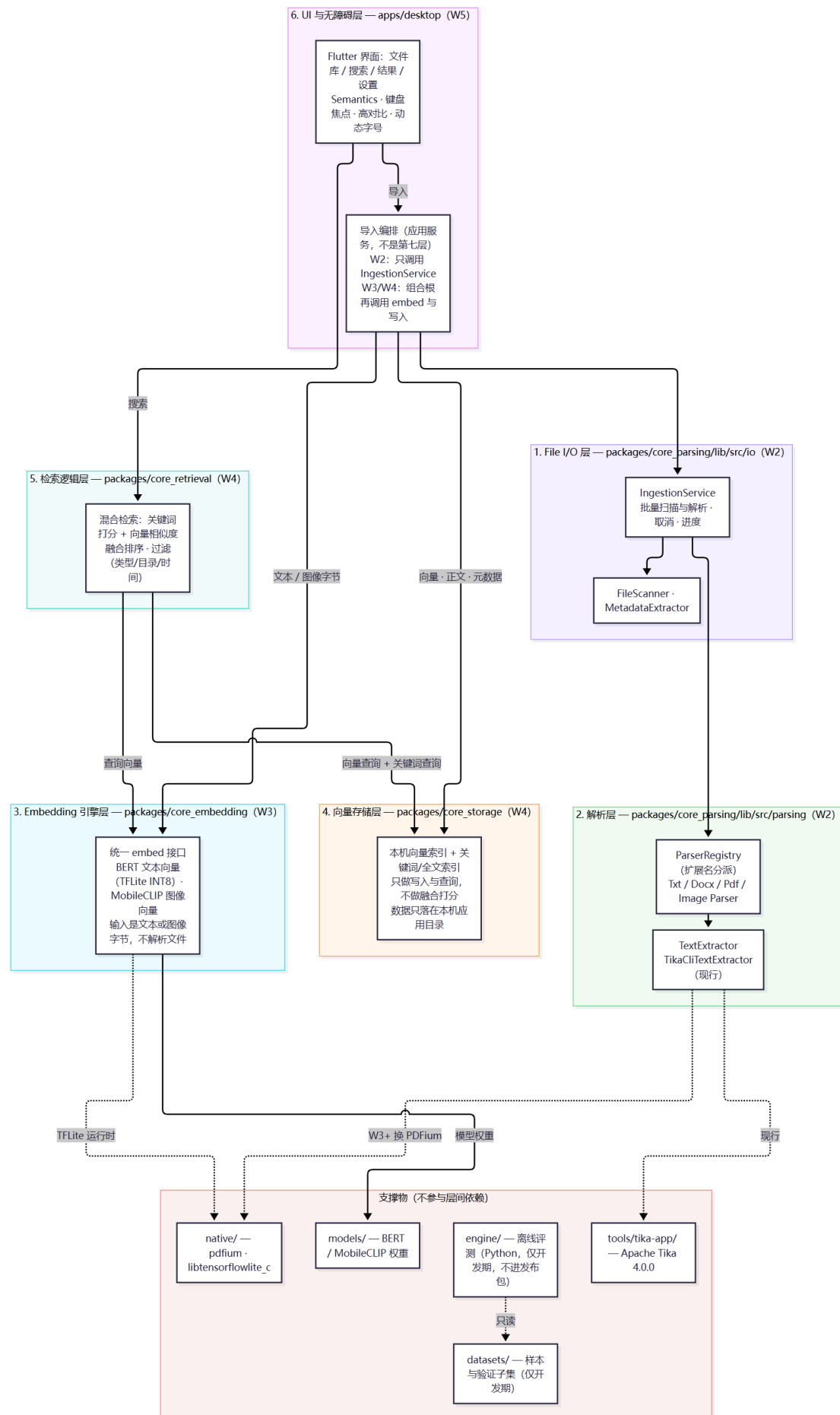
1.2 架构级硬约束

编号	约束	架构影响
C1	完全离线 ：导入、检索、打开本地文件都不联网，默认不外传任何用户数据	不允许任何 model/telemetry 下载；模型与依赖全部随包分发或本机已有
C2	跨平台 ：Windows（开发主平台）、macOS、Linux	禁止平台私有 API 直连；进程调用、路径、classpath 分隔符必须平台分支
C3	WCAG 2.1 AA ：键盘走完全流程、控件有名称、高对比、字号放大	每个长任务必须可取消 + 可播报进度；状态不得只靠颜色
C4	离线优先的性能 ：批量导入不假死、可取消；已建库后检索要快	重活不能占 UI 线程；解析层必须支持取消与进度
C5	可测性 ：核心模块单测覆盖率最终 ≥90%（W2 解析模块 ≥80%）	文件系统、外部进程、模型推理一律经接口注入，便于打桩
C6	开源合规 ：Apache 2.0	依赖许可证必须在 W7 逐项核对；native 二进制与模型权重单独记录来源

2. 架构原则

1. **单向依赖**：上层依赖下层，下层不知道上层的存在；跨层只经接口，不 import 实现。
 2. **纯逻辑与副作用分离**：解析、打分、切块是可测的纯逻辑；文件读写、进程、模型推理是副作用，藏在 `spi/` (Service Provider Interface) 后面。
 3. **失败隔离**：一个坏文件不能中断一批文件；错误用 `ParseFailure`，不用异常穿层。
 4. **契约先行**：接口在实现之前冻结 (W2 Day 1 完成)，实现按接口填充。
 5. **离线即默认**：任何"要联网才能跑"的设计都视为缺陷，而不是配置问题。
-

3. 六层架构图



实线是调用方向，含尚未落地的 W3–W5。虚线是可替换实现或开发期数据，不进入层间依赖。

导入和搜索分开走。导入编排在 `apps/desktop` 的组合根：W2 只调用 `IngestionService`（扫描 + 解析）；W3/W4 再依次把文本或图像字节送给 `Embedding`、把向量和正文写入存储。`core_parsing` 不依赖上层包。检索层同时调用 `Embedding` 和存储层，这两层互不依赖。关键词索引进存储层，融合打分留在检索层。L1、L2 标 W2 表示本周范围；接口已冻结。

3.1 各层职责与边界

层	职责（做什么）	明确不做	周次
1. File I/O	目录递归扫描、白名单过滤、隐藏/临时文件跳过、stat 元数据、内容哈希； <code>IngestionService</code> 编排扫描和解析，带进度与取消	不解析正文、不调用嵌入、不写索引	W2
2. Parsing	按扩展名分派；TXT/PDF/DOCX/JPG/PNG → 文本分页 + 图像元数据 + <code>ParseOutcome</code>	不做向量化、不做 OCR、不做 UI 文案	W2
3. Embedding Engine	统一 <code>embed()</code> ；文本走 BERT(TFLite)、图像走 MobileCLIP；批处理。输入由导入编排传入	不解析文件、不知道向量存哪	W3
4. Vector Storage	本地持久化向量索引和关键词/全文索引、查询、按元数据删除	不打分、不排序、不融合关键词与向量	W4
5. Retrieval Logic	同时取查询向量和两类索引，做混合打分、排序、过滤、Top-K	不碰文件系统、不直接读原始文件、不写索引	W4
6. UI 与无障碍	四个界面、Semantics、焦点顺序、高对比、字号；导入编排在应用服务里依次调用下层	Widget 不读文件、不算相似度、不直接写索引	W5

3.2 横切关注点

关注点	归属	约定
错误模型	各级共用的 <code>model/</code> 值对象	用 <code>ParseFailure</code> / 返回码表达可预期失败；异常只用于编程错误
取消	<code>spi/CancellationToken</code>	协作式；长任务在安全点调用 <code>throwIfCancelled()</code>
并发调度	<code>spi/Executor</code>	W2 用 <code>ImmediateExecutor</code> ；W6 可换成 <code>worker isolate</code> ，调用方不改
外部进程	<code>spi/ProcessRunner</code>	全包唯一允许 <code>spawn</code> 进程的位置
日志	(W7 统一)	绝不记录文件正文；只记路径、大小、耗时、错误类型

4. 仓库目录映射

```
accessible-multimodal-retrieval/
├─ apps/desktop/
│   └─ pubspec.yaml
├─ packages/
│   └─ core_parsing/
│       ├── lib/core_parsing.dart
│       ├── lib/src/model/
│       └── lib/src/spi/
TextExtractor

# 6. UI 与无障碍层 (Flutter, W5)：导入编排在组合根
#   W2: 以 path 依赖引入 core_parsing

# 1+2. File I/O + Parsing (W2)
#   唯一公开入口 (barrel)
#   跨层值对象
#   注入缝: ProcessRunner / Executor / CancellationToken /
```

lib/src/parsing/	# DocumentParser + ParserRegistry
parsers/	# txt / docx / pdf / image
adapters/	# TikaCliTextExtractor
lib/src/io/	# FileScanner / MetadataExtractor / IngestionService
tool/	# 夹具生成等开发脚本
test/	# 单元测试 + fixtures
core_embedding/	# 3. Embedding 引擎 (W3)
core_storage/	# 4. 向量存储 (W4)
core_retrieval/	# 5. 检索逻辑 (W4)
native/	# FFI 产物 (不入库二进制, 只留结构与 README)
pdfium/{windows,include,tests}	# PDFium 动态库与头文件预留位
tflite/windows/	# libtensorflowlite_c 预留位
gtest/	# Google Test (原生单元测试)
engine/	# 离线评测脚本 (Python + Chroma, 仅开发期, 不进发布包)
datasets/	# 样本 + 验证子集 (图片按 .gitignore 不入库)
models/	# 模型权重 (.gitignore 排除实体文件)
tools/tika-app/	# Apache Tika 4.0.0 发行版 (jar 不入库)
docs/	# 架构 / API / TDD / 任务文档
reports/	# 每周交付报告与证据

4.1 「新增一种格式要改哪几个文件」

步骤	文件	说明
1	lib/src/parsing/parsers/xxx_parser.dart	实现 DocumentParser，只做纯逻辑
2	lib/src/parsing/parser_registry.dart 的装配处	注册一行：扩展名 → parser
3	test/fixtures/ + tool/make_fixtures.py	加夹具（含一个损坏样本）
4	test/parsing/xxx_parser_test.dart	正常 / 损坏 / 边界三类用例

其余四层无需改动 —— 这是本架构对"模块化、可维护"的可验证回答。

5. 依赖方向与跨层契约

导入：

```

UI → 导入编排
    | IngestionService
    |   | FileScanner / MetadataExtractor
    |   |   | Parsing → TextExtractor
    | Embedding           (文本 / 图像字节)
    | Storage             (向量 · 正文 · 元数据)
```

搜索：

```

UI → Retrieval
    | Embedding           (查询向量)
    | Storage             (向量查询 + 关键词查询)
```

规则：

- **R1** 下层不得依赖上层。core_parsing 不 import core_embedding、core_storage、core_retrieval。
- **R1a** 导入编排在应用组合根，可以依次调用 File I/O、Parsing、Embedding、Storage。Embedding 不调用 Parsing，也不调用 Storage；写入由编排者完成。
- **R1b** 检索层同时依赖 Embedding 与 Storage。Storage 不调用 Embedding。

- **R2** 层与层之间传 `ParsedDocument / FileMetadata / IngestRecord` 这类**不可变值**，不传文件句柄、不传 `File` 对象。交给 `Embedding` 的是文本或图像字节，交给 `Storage` 的是向量、正文和元数据。
- **R3** 副作用（文件、进程、模型、存储）必须经 `spi/` 接口注入；`dart:io` 的 `Process` 只允许出现在 `spi/` 的实现里。
- **R4** `Widget` 不得 `import 'dart:io'` 读文件。导入由应用服务编排，搜索只消费检索层返回的结果。

6. 冻结的公共接口（Week 2）

入口： `packages/core_parsing/lib/core_parsing.dart`（`lib/src/` 下均为实现细节）。

6.1 值对象（`lib/src/model/`）

类型	作用	关键成员
<code>ParseOutcome</code>	解析/入库的分类词汇表	<code>ok / partial / skipped / failed</code> , <code>isUsable</code>
<code>DocumentPage</code>	一页文本	<code>pageNumber</code> （1 起）、 <code>text</code> 、 <code>isEmpty</code> 、 <code>charCount</code>
<code>ImageAsset</code>	图像描述（W2 只到元数据）	<code>format</code> 、 <code>width</code> 、 <code>height</code> 、 <code>colorMode</code> 、 <code>pixelCount</code> 、 <code>aspectRatio</code>
<code>ParsedDocument</code>	一个文件的解析产物	<code>fullText</code> 、 <code>pages</code> 、 <code>imageAsset</code> 、 <code>attributes</code> 、 <code>outcome</code> 、 <code>warnings</code> 、 <code>pageCount</code> 、 <code>hasText</code> 、 <code>isImageOnly</code>
<code>ParseFailure</code>	结构化失败	<code>kind</code> 、 <code>message</code> 、 <code>detail</code> 、 <code>outcome</code> （ <code>kind</code> → <code>outcome</code> 的降级策略）
<code>ParseResult</code>	解析结果联合体	<code>document / failure / duration / outcome / isSuccess</code>
<code>FileMetadata</code>	文件元数据	<code>id</code> 、 <code>path</code> 、 <code>relativePath</code> 、 <code>extension</code> 、 <code>sizeBytes</code> 、 <code>mimeType</code> 、 <code>createdAt/modifiedAt/accessedAt</code> 、 <code>contentHash</code> 、 <code>parserName</code> 、 <code>status</code> 、 <code>imageAsset</code>
<code>IngestRecord</code>	单文件入库记录	<code>metadata</code> 、 <code>document</code> 、 <code>failure</code> 、 <code>outcome</code> 、 <code>hasContent</code>
<code>IngestionReport</code>	一次批量入库汇总	<code>records</code> 、 <code>succeeded/partial/skipped/failed</code> 、 <code>failures</code> 、 <code>indexable</code> 、 <code>isClean</code> 、 <code>cancelled</code>
<code>IngestProgress</code>	进度通知（无障碍播报用）	<code>processed</code> 、 <code>total</code> 、 <code>currentPath</code> 、 <code>elapsed</code> 、 <code>fraction</code>

所有值对象均为 `@immutable`，实现 `== / hashCode / toString / toJson()`。

6.2 接口（`lib/src/parsing/`、`lib/src/spi/`）

```
abstract interface class DocumentParser {
  String get name; // 'txt' | 'docx' | 'pdf' | 'image'
  String get version; // 解析行为变化就 +1，保证基准可比
  Set<String> get supportedExtensions; // {'.pdf'}，小写带点
  Future<ParseResult> parse(String absolutePath, {CancellationToken? cancel});
}

abstract interface class ParserRegistry {
  List<DocumentParser> get parsers;
```

```
Set<String> get supportedExtensions;
DocumentParser? resolveForExtension(String extension);
DocumentParser? resolveForPath(String absolutePath);
bool supports(String absolutePath);
}

abstract interface class TextExtractor {
  String get name;
  String get version;
  Future<ExtractedText> extract(String absolutePath, {CancellationToken? cancel});
}

abstract interface class ProcessRunner {
  Future<ProcessRunResult> run(String executable, List<String> arguments,
    {String? workingDirectory, Map<String, String>? environment,
    Duration? timeout, CancellationToken? cancel});
}

abstract interface class Executor {
  Future<T> run<T>(Future<T> Function() action);
}

class CancellationToken { bool get isCancelled; void throwIfCancelled(); ... }
class CancellationTokenSource { CancellationToken get token; void cancel(); }
```

6.3 契约条款（实现者必须遵守）

条款	内容
P1	DocumentParser.parse 不得因文件本身的问题抛异常；损坏/不支持/缺依赖一律返回 ParseResult.failure
P2	解析耗时必须写入 ParseResult.duration（性能基准要用）
P3	长任务在每个文件边界与每个分页边界检查 cancel.throwIfCancelled()
P4	只有 ProcessRunner 的实现可以 Process.run；解析器只依赖接口
P5	外部进程的 stdout 才是正文，stderr 只作诊断（Tika 会往 stderr 打 INFO 噪声）
P6	分页：PDF 一页一条 DocumentPage；TXT/DOCX 单页；图像无文本
P7	编码：TXT 必须支持 UTF-8 / UTF-8 BOM / UTF-16 LE-BE / GBK，降级时写 warnings（如 encoding-fallback:gbk）
P8	任何日志/报告不得包含文件正文（隐私约束 C1）

7. 三层工程结构（包 / 应用 / 支撑）

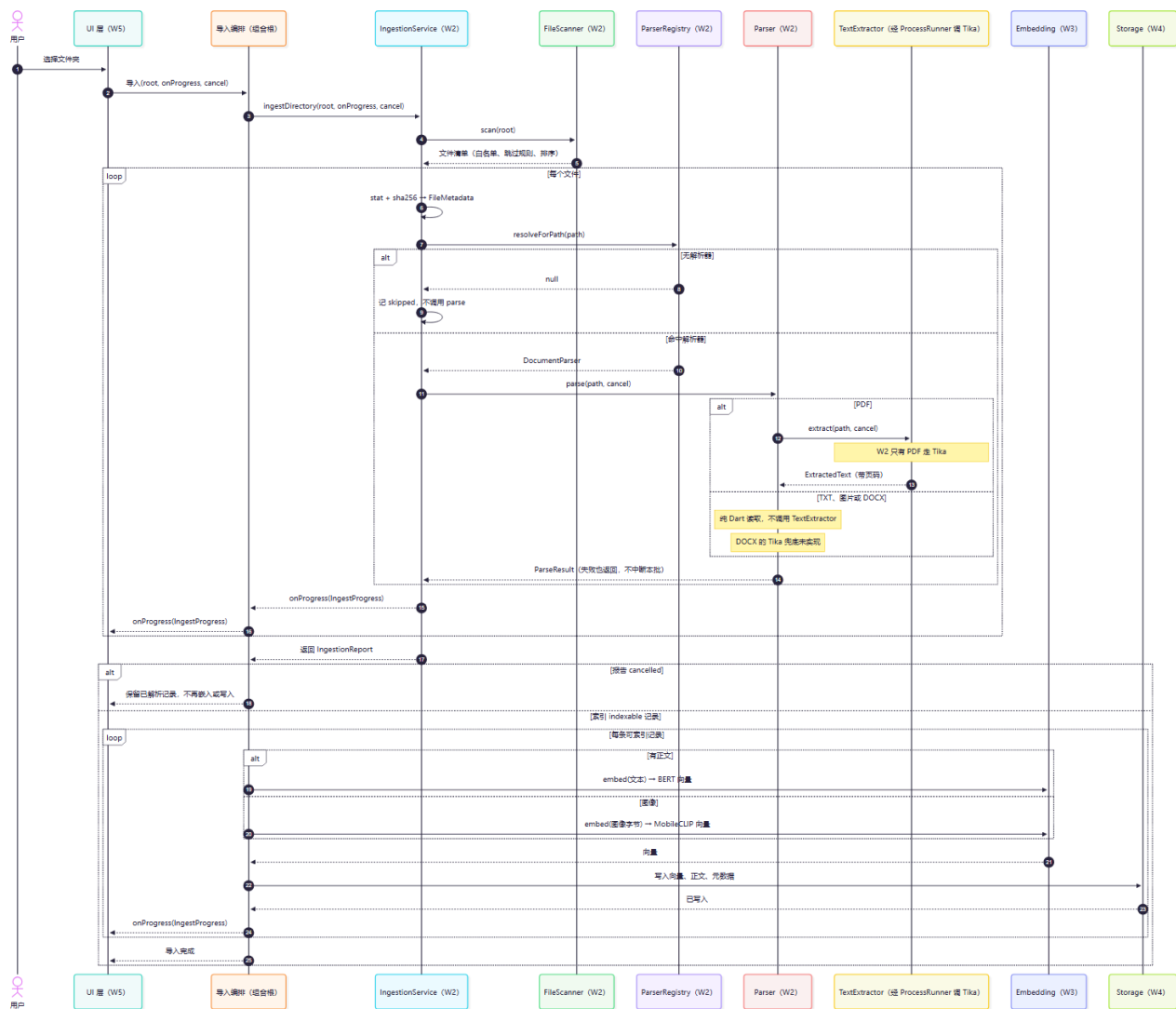
类型	内容	是否进入发布包
Dart 包	packages/core_*	是（编译进 Flutter 应用）
Flutter 应用	apps/desktop	是
native 产物	native/pdfium、native/tflite	是（动态库随包）
开发期脚本	engine/（Python 评测）、tools/tika-app（PDF 解析兜底）	engine/ 否；tools/tika-app 见 ADR 0002

类型	内容	是否进入发布包
数据集与权重	datasets/、models/	样本随包；验证子集与权重不入仓库

8. 端到端数据流

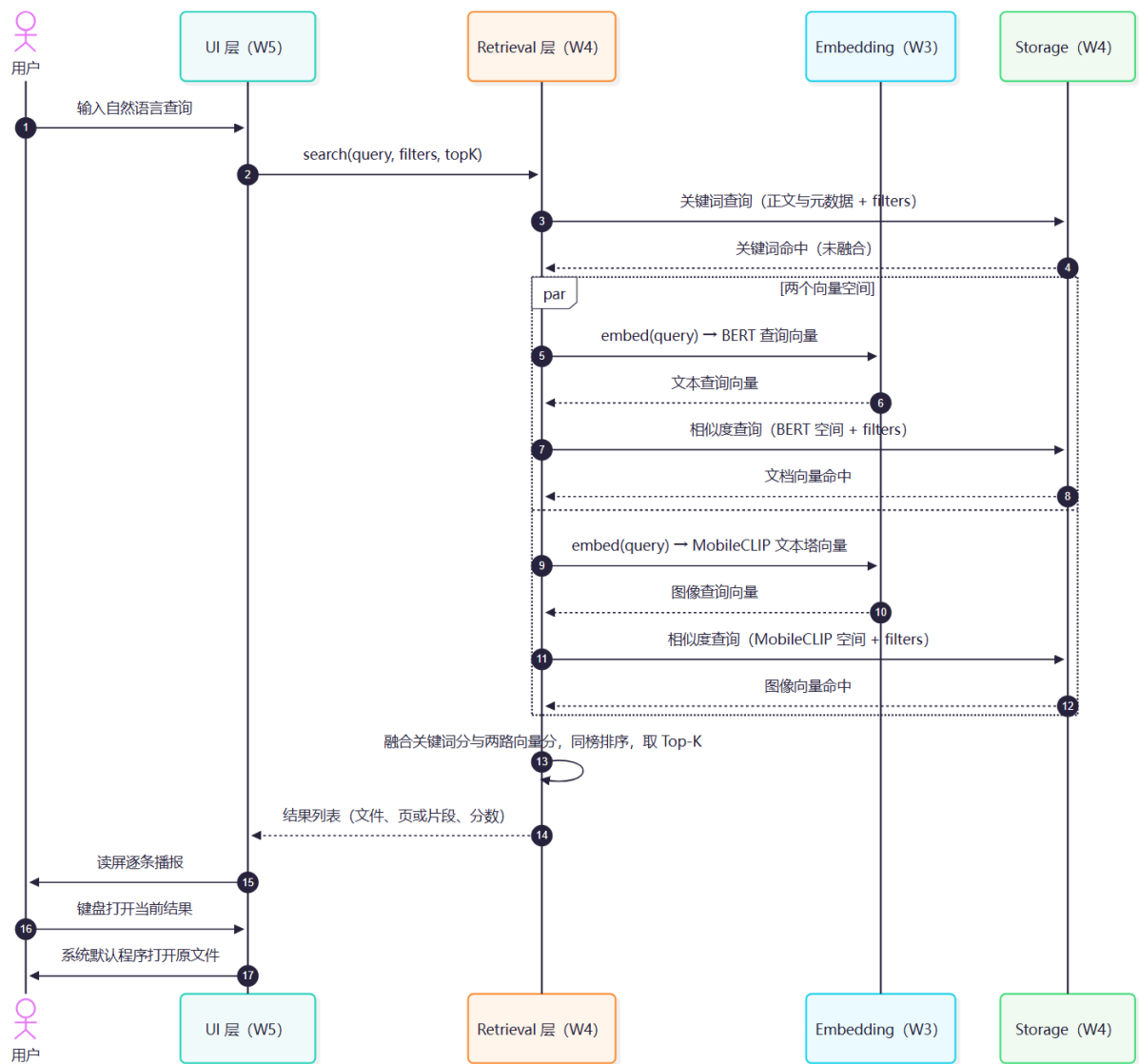
8.1 索引（摄入）流程

W2 收到 IngestionReport 即停。嵌入和写入在 W3/W4 由导入编排按 indexable 逐条调用，不进入 IngestionService。



8.2 检索流程 (W4/W5 目标态)

关键词查询不依赖向量。文档和图像分属 BERT 与 MobileCLIP 两个空间，查询要做两次 `embed(query)`。存储只返回命中，融合在检索层。返回单位写「页或片段」，切块策略仍见 TDD 的 Q3。



9. 进程 / 线程 / 隔离模型

执行体	承载内容	W2 状态	后续
Flutter UI isolate	界面、事件、取消源	未接入	W5
主 isolate (Dart)	扫描、编排、TXT/DOCX/图像解析、元数据	W2 落地	纯 CPU 部分 (XML/图像解码) 在 W6 视实测结果移入 worker isolate
worker isolate	批量解析 (可选)	接口已预留 (Executor)	W6
外部进程	java + Tika, 只服务 PDF	W2 落地	W3+ 由 PDFium FFI 替代以去掉 JVM

10. 错误分层与降级策略

失败场景	表达	分类	对整批的影响
扩展名无解析器	ParseFailure(unsupportedFormat)	skipped	跳过, 计数
文件权限不足	ParseFailure(permissionDenied)	skipped	跳过, 计数
文件损坏 / 非声明格式	ParseFailure(corruptedInput)	failed	跳过该文件, 继续批
缺 JDK (PDF 路径)	ParseFailure(dependencyUnavailable)	failed	跳过该文件; 报告单列, 用户可修环境后重跑
外部进程超时	ParseFailure(timeout)	failed	跳过该文件
文件超过大小上限	ParsedDocument(outcome: partial) + warnings	partial	入库但标记截断
编码无法识别	降级为 GBK/latin1 + warnings	ok/partial	正常入库
用户取消	CancelledException 冒泡到编排层	报告 cancelled=true	停止处理剩余文件, 已完成的记录保留
编程错误 (bug)	抛异常	—	必须修, 不允许靠 catch 掩盖

failed 表示"这个文件不行", skipped 表示"我们没打算处理它"。报告把两者分开, 避免把正常的格式过滤当成缺陷 (W2 覆盖率与报告口径都以此为准)。

11. 安全与隐私 (架构级)

要求	实现方式
不联网	运行时不发起任何网络请求; 模型与依赖随包或本机已存在 (C1)
不外传	无遥测、无崩溃上报、无云同步
数据落地位置	索引数据只写本机应用目录 (W4 定义具体路径), 提供"清除索引"

要求	实现方式
日志	只记路径/大小/耗时/错误类型；永不记录文件正文（契约 P8）
仓库卫生	.gitignore 排除 .env/、chroma_data/、模型权重、非 samples 的数据集图片（对应风险 R9）
进程调用	只调用本机已知可执行文件（java），参数为固定模板 + 文件路径，不做 shell 拼接

12. 无障碍的前置架构约束（WCAG 2.1 AA 对齐）

- 1. **可播报的进度**：IngestProgress 强制同时携带 processed 与 total（不允许只有百分比），UI 才能缝合完整句子给读屏。
- 2. **可取消**：所有长任务（扫描、解析、后续的嵌入与检索）都接受 CancellationToken，键盘用户必须能中止而不是干等。
- 3. **状态可文本化**：ParseOutcome 与 ParseFailure.kind 是可枚举的结构化值，UI 可翻译成"成功/部分内容/已跳过/失败"，不依赖颜色区分。
- 4. **结果可被读屏逐项消费**：FileMetadata 自带 fileName、relativePath、mimeType、modifiedAt，结果列表无需再次访问文件系统即可播报。

13. 测试策略与覆盖率口径

层级	手段	W2 目标
逻辑层 (lib/)	dart test 单测，注入假 ProcessRunner / 夹具文件	行覆盖率 ≥80%
外部集成 (真 Java、真 PDF/DOCX)	集成测试读 datasets/samples/，标记 @Tags(['integration'])	冒烟通过即可
UI	flutter test (W5 起)	—
原生	Google Test (W3 起, native/gtest/)	—
覆盖率口径	只统计 packages/core_parsing/lib/；coverage/lcov.info 的 LF/LH 汇总	证据入 reports/week2/

不允许用"跑真 Java"来凑覆盖率：外部进程一律经 ProcessRunner 打桩（契约 P4）。

14. 决策记录（ADR）索引

编号	决策	状态
ADR 0001	解析模块做成独立纯 Dart 包 packages/core_parsing	已采纳
ADR 0002	PDF 文本抽取先用 Apache Tika CLI，PDFium FFI 后续替换	已采纳（带迁移触发条件）
ADR 0003 (待写, W4)	向量存储选型：Chroma Dart FFI vs SQLite + 近邻	未决
ADR 0004 (待写, W6)	是否把批量解析移入 worker isolate	未决

覆盖率报告

命令

在 packages/core_parsing 下执行。真 Java、以及读取被 gitignore 的 coco / rvlcdip 图片的测试带 integration 标签，不计入这次覆盖率。

```
dart test --coverage=coverage --exclude-tags integration
dart run coverage:format_coverage --lcov --in=coverage --out=coverage/lcov.info --report-on=lib
```

附录 C 对 reports/week2/coverage/lcov.info 的统计：

```
lines hit 949 / 1043 = 90.99%
```

同一数字写在 reports/week2/logs/coverage.txt。

证据

文件	内容
reports/week2/coverage/lcov.info	format_coverage --report-on=lib 的副本
reports/week2/logs/coverage.txt	两条命令和百分比
reports/week2/screenshots/coverage-percent.png	终端中覆盖率百分比显示

解析模块 API

1. 调用约定

外部代码 import:

```
import 'package:core_parsing/core_parsing.dart';
```

一次目录入库:

```
final IngestionReport report = await IngestionService(
  registry: StandardParserRegistry(<DocumentParser>[
    TxtParser(),
    const DocxParser(),
    PdfParser(
      textExtractor: TikaCliTextExtractor(
        processRunner: const IoProcessRunner(),
        tikaHome: r'tools\tika-app',
      ),
    ),
    const ImageParser(),
  ]),
  metadata: FileMetadataExtractor(),
).ingestDirectory(r'datasets\samples');
```

异常。 文件损坏、缺 Java、超时、权限不足、不支持的扩展名都变成 `ParseFailure`，放进 `ParseResult` 或 `IngestRecord`。`CancelledException` 是唯一会从解析器冒出来的协作式取消。`IngestionService.ingestDirectory` 接住扫描阶段的 `FileSystemException` 和 `CancelledException`，返回报告。`maxDepth < 0` 以及构造参数断言失败仍抛异常，那是调用错误。

线程。 全部实现都在调用方的 `isolate` 上运行。`ImmediateExecutor` 不把工作送进别的 `isolate`。这些对象没有内部锁。`CancellationToken` 可以在同一个 `isolate` 里跨回调使用，不要把它传到另一个 `isolate`。

`IoProcessRunner` 通过 `dart:io` 收子进程输出，回调发生在当前 `isolate` 的事件循环上。

2. 结果

`ParseOutcome`

```
enum ParseOutcome { ok, partial, skipped, failed }
bool get isUsable

const ParseOutcome outcome = ParseOutcome.ok;
// outcome.isUsable == true
// ParseOutcome.skipped.isUsable == false
```

`ok` 是完整成功。`partial` 是成功但有截断或降级。`skipped` 是跳过（格式不支持、权限不足、取消前就停）。`failed` 是试过之后失败。`isUsable` 只在 `ok` 和 `partial` 时为真。

`ParseFailureKind` / `ParseFailure`

```
enum ParseFailureKind {
  unsupportedFormat,
  corruptedInput,
  dependencyUnavailable,
  timeout,
  permissionDenied,
  cancelled,
  unknown,
}

class ParseFailure {
  const ParseFailure({required this.kind, required this.message, this.detail});
  ParseOutcome get outcome;
  Map<String, Object?> toJson();
}

const ParseFailure failure = ParseFailure(
  kind: ParseFailureKind.corruptedInput,
  message: 'PDF is empty.',
);
// failure.outcome == ParseOutcome.failed
```

`unsupportedFormat`、`permissionDenied`、`cancelled` 映射到 `skipped`。其余映射到 `failed`。`message` 给人看，`detail` 放 `stderr` 或异常文本，不放正文。`toJson` 含 `kind`、`outcome`、`message`、`detail`。不抛异常。

ParsedDocument

```
class ParsedDocument {
  const ParsedDocument({
    required this.fullText,
    this.pages = const <DocumentPage>[],
    this.imageAsset,
    this.attributes = const <String, Object?>{},
    this.outcome = ParseOutcome.ok,
    this.warnings = const <String>[],
  });
  bool get hasText;
  int get pageCount;
  bool get isImageOnly;
  int get charCount;
  Map<String, Object?> toJson();
}

const ParsedDocument document = ParsedDocument(fullText: 'Hello\n');
// document.pageCount == 1
// document.hasText == true
const ParsedDocument blank = ParsedDocument(fullText: ' ');
// blank.pageCount == 0
```

没有 pages 时，有正文记 1 页，只有空白记 0 页。isImageOnly 表示有图且没有正文。warnings 例如 encoding-fallback:gbk。不抛异常。

ParseResult

```
class ParseResult {
  const ParseResult.success(ParsedDocument document, {Duration duration = Duration.zero});
  const ParseResult.failure(ParseFailure failure, {Duration duration = Duration.zero});
  ParseOutcome get outcome;
  bool get isSuccess;
  bool get isFailure;
  Map<String, Object?> toJson();
}

const ParseResult ok = ParseResult.success(
  ParsedDocument(fullText: 'Hello\n'),
);
const ParseResult bad = ParseResult.failure(
  ParseFailure(kind: ParseFailureKind.unknown, message: 'missing'),
);
// ok.isSuccess == true
// bad.document == null
// bad.failure?.outcome == ParseOutcome.failed
```

成功时 failure 为 null，失败时 document 为 null。duration 是这一次解析的墙钟时间。

DocumentPage

```
class DocumentPage {
  const DocumentPage({required this.pageNumber, required this.text});
  bool get isEmpty;
  int get charCount;
```

```

    Map<String, Object?> toJson();
}

const DocumentPage page = DocumentPage(pageNumber: 1, text: '第一页');
// page.charCount == 3
// page.isEmpty == false

```

pageNumber 从 1 开始。TXT 和 DOCX 只有第 1 页。PDF 一页一条。

ImageAsset

```

class ImageAsset {
  const ImageAsset({
    required this.format,
    required this.width,
    required this.height,
    this.colorMode,
    this.orientation,
  });
  int get pixelCount;
  double get aspectRatio;
  Map<String, Object?> toJson();
}

const ImageAsset asset = ImageAsset(
  format: 'png',
  width: 1186,
  height: 1137,
  colorMode: 'rgb',
);
// asset.pixelCount == 1186 * 1137
// asset.orientation == null

```

format 是小写容器名，本包写出 png 或 jpeg。colorMode 为 rgb、rgba、grayscale、grayscale-alpha。没有 EXIF 时 orientation 为 null。高度为 0 时 aspectRatio 为 0。

3. 文件与入库

FileMetadata

```

class FileMetadata {
  const FileMetadata({
    required this.id,
    required this.path,
    required this.relativePath,
    required this.fileName,
    required this.extension,
    required this.sizeBytes,
    this.mimeType,
    this.createdAt,
    this.modifiedAt,
    this.accessedAt,
    this.contentHash,
    this.indexedAt,
    this.parserName,
    this.parserVersion,
  });
}

```

```

    this.parseDurationMs,
    this.status = ParseOutcome.skipped,
    this.errorMessage,
    this.imageAsset,
  });
  bool get isIndexable;
  FileMetadata copyWith({/* 与字段同名的可选参数 */});
  Map<String, Object?> toJson();
}

```

```

const FileMetadata metadata = FileMetadata(
  id: 'abc',
  path: r'datasets\samples\sample.txt',
  relativePath: 'sample.txt',
  fileName: 'sample.txt',
  extension: '.txt',
  sizeBytes: 12,
  status: ParseOutcome.ok,
);
// metadata.isIndexable == true

```

id 有内容哈希时是 SHA-256，否则是相对路径的哈希。relativePath 用 /。createdAt 来自 stat.changed，因为 dart:io 没有 birth time。isIndexable 等于 status.isUsable。copyWith 只替换传入的字段。

IngestRecord / IngestionReport / IngestProgress

```

class IngestRecord {
  const IngestRecord({required this.metadata, this.document, this.failure});
  ParseOutcome get outcome;
  bool get hasContent;
  Map<String, Object?> toJson({bool includeText = false});
}

```

```

class IngestionReport {
  const IngestionReport({
    required this.rootPath,
    required this.startedAt,
    required this.finishedAt,
    this.records = const <IngestRecord>[],
    this.cancelled = false,
  });
  Duration get duration;
  int get total;
  int countOf(ParseOutcome outcome);
  int get succeeded; int get partial; int get skipped; int get failed;
  List<IngestRecord> get failures;
  List<IngestRecord> get indexable;
  bool get isClean;
  Map<String, Object?> toJson({bool includeRecords = true});
}

```

```

class IngestProgress {
  const IngestProgress({
    required this.processed,
    required this.total,
    this.currentPath,
    this.elapsed = Duration.zero,
  });
}

```

```

    double get fraction;
    Map<String, Object?> toJson();
}

final IngestRecord record = IngestRecord(
    metadata: FileMetadata(
        id: 'abc',
        path: r'datasets\samples\sample.txt',
        relativePath: 'sample.txt',
        fileName: 'sample.txt',
        extension: '.txt',
        sizeBytes: 12,
        status: ParseOutcome.ok,
    ),
);
final IngestionReport report = IngestionReport(
    rootPath: r'datasets\samples',
    startedAt: DateTime.utc(2026, 10, 5),
    finishedAt: DateTime.utc(2026, 10, 5, 0, 0, 1),
    records: <IngestRecord>[record],
);
const IngestProgress progress = IngestProgress(processed: 1, total: 2);
// record.toJson() 不含正文, 只留 charCount
// progress.fraction == 0.5
// report.isClean == true

```

toJson(includeText: false) 不复制正文, 只留长度。isClean 表示没有 failed 且没有取消。fraction 在 total <= 0 或已经做完时为 1。

defaultIngestExtensions / FileScanner

```

const Set<String> defaultIngestExtensions;
class FileScanner {
    const FileScanner();
    Future<List<String>> scan(
        String rootPath, {
        Set<String> extensions = defaultIngestExtensions,
        bool followLinks = false,
        int maxDepth = 32,
        CancellationToken? cancel,
    });
}

const FileScanner scanner = FileScanner();
final List<String> files = await scanner.scan(r'datasets\samples');
// files 是已排序的绝对路径, 扩展名落在 defaultIngestExtensions 里

```

白名单默认是 .txt、.pdf、.docx、.jpg、.jpeg、.png。扩展名可以不带点, 比较时忽略大小写。跳过 . 或 ~\$ 开头的名字, 以及 \$Recycle.Bin、System Volume Information。结果是绝对路径, 已排序。maxDepth 为 0 只看根目录自己的文件。

抛出异常: maxDepth < 0 抛 ArgumentError; 根不存在或不是目录抛 FileSystemException; 已取消抛 CancelledException。入库服务会把后两种收成报告, 直接调用扫描器则不会。

MetadataExtractor / FileMetadataExtractor / mimeTypeForExtension

```
abstract interface class MetadataExtractor {
    Future<FileMetadata> describe(
        String absolutePath, {
            required String rootPath,
            int? maxFileSizeBytes,
            CancellationToken? cancel,
        });
}

class FileMetadataExtractor implements MetadataExtractor {
    FileMetadataExtractor({this.maxFileSizeBytes = 10 * 1024 * 1024});

    String? mimeTypeForExtension(String extension);

    final FileMetadataExtractor extractor = FileMetadataExtractor();
    final FileMetadata described = await extractor.describe(
        r'datasets\samples\sample.png',
        rootPath: r'datasets\samples',
    );
    // mimeTypeForExtension('.png') == 'image/png'
    // mimeTypeForExtension('.bin') == null
}
```

超过大小上限的文件仍返回元数据，但 `contentHash` 为 `null`。权限错误的 `status` 是 `skipped`，其他读失败是 `failed`，两者都不抛 `FileSystemException`。已取消则抛 `CancelledException`。`maxFileSizeBytes` 必须为正，否则构造时断言失败。

`mimeTypeForExtension` 认识 `.txt`、`.pdf`、`.docx`、`.jpg`、`.jpeg`、`.png`，其余返回 `null`。不抛异常。

IngestionService

```
class IngestionService {
    IngestionService({
        required this.registry,
        required this.metadata,
        this.scanner = const FileScanner(),
        this.executor = const ImmediateExecutor(),
        this.concurrency = 2,
        this.maxFileSizeBytes = 10 * 1024 * 1024,
    });

    Future<IngestionReport> ingestDirectory(
        String rootPath, {
            Set<String> extensions = defaultIngestExtensions,
            bool followLinks = false,
            int maxDepth = 32,
            void Function(IngestProgress)? onProgress,
            CancellationToken? cancel,
        });
}
```

调用示例见第 1 节。

顺序是扫描、逐文件描述、按 SHA-256 去重、解析、汇总。一个坏文件记入报告后继续。相同内容的第二个文件为 `skipped`，不再解析。`concurrency` 是同时处理的文件数，默认执行器仍在当前 `isolate`。
`concurrency >= 1, maxFileSizeBytes > 0`，否则构造时断言失败。

根目录不存在、不是目录或列不出来：返回一条记录，权限不足为 `skipped`，其余为 `failed`，不抛 `FileSystemException`。调用前已经取消：空报告且 `cancelled` 为真，并回调一次 `0/0`。扫描或单个解析器抛出 `CancelledException`：停掉剩余文件，已完成的记录保留。

4. 解析器

DocumentParser

```
abstract interface class DocumentParser {
    String get name;
    String get version;
    Set<String> get supportedExtensions;
    Future<ParseResult> parse(String absolutePath, {CancellationToken? cancel});
}

final DocumentParser parser = TxtParser();
final ParseResult parsed = await parser.parse(r'test\fixtures\hello.txt');
// 损坏或缺失返回 ParseResult.failure，不向外抛文件错误
```

文件本身的问题必须返回 `ParseResult.failure`。实现应在读取前调用 `cancel.throwIfCancelled()`，那时抛 `CancelledException`。不要为损坏文件抛其他异常；万一抛了，入库服务会记成 `unknown`。

ParserRegistry / StandardParserRegistry / normalizeParserExtension

```
abstract interface class ParserRegistry {
    List<DocumentParser> get parsers;
    Set<String> get supportedExtensions;
    DocumentParser? resolveForExtension(String extension);
    DocumentParser? resolveForPath(String absolutePath);
    bool supports(String absolutePath);
}

class StandardParserRegistry implements ParserRegistry {
    StandardParserRegistry(Iterable<DocumentParser> parsers);
    Future<ParseResult> parse(String absolutePath, {CancellationToken? cancel});
}

String? normalizeParserExtension(String extension);

final StandardParserRegistry registry = StandardParserRegistry(<DocumentParser>[
    TxtParser(),
]);
final DocumentParser? parser = registry.resolveForPath(r'notes\A.TXT');
// parser != null
// normalizeParserExtension('TXT') == '.txt'
// normalizeParserExtension('.') == null
```

同一个扩展名第一次注册的解析器生效。未知扩展名返回 `null`，`parse` 返回 `unsupportedFormat`，不抛异常。`normalizeParserExtension` 把 `TXT`、`.TXT`、`txt` 都收成 `.txt`；空串和单独的 `.` 返回 `null`。不抛异常。

TxtParser

```
class TxtParser implements DocumentParser {
    TxtParser({this.maxBytes = 10 * 1024 * 1024});
    static const int defaultMaxBytes = 10 * 1024 * 1024;
}

final ParseResult result = await TxtParser().parse(r'test\fixtures\hello.txt');
// result.outcome == ParseOutcome.ok
// result.document?.fullText 含归一化后的换行
```

嗅探顺序: UTF-8 BOM、UTF-16 LE/BE BOM、严格 UTF-8, 最后 GBK。换行收成 \n。超过 maxBytes 时 outcome 为 partial, 并尽量不把半个字符留给 GBK。GBK 兜底会在 warnings 里写 encoding-fallback:gbk。空文件是 ok 加空正文。缺失文件是 failed。maxBytes 必须为正。取消抛 CancelledException。

DocxParser

```
class DocxParser implements DocumentParser {
    const DocxParser();
}

final ParseResult result = await const DocxParser().parse(
    r'datasets\samples\sample.docx',
);
// 成功时 outcome 为 ok, 正文非空, 且不调用 Tika
```

纯 Dart: 解 word/document.xml, 按段落拼接, w:tab 成制表符, w:br 成换行。docProps/core.xml 里有值才写入 dc:title、dc:creator、dcterms:created。没有该部件时正文仍然成功, 这三个键不出现。坏 zip、缺 word/document.xml、XML 无法解析都是 corruptedInput / failed。不调用 Tika。取消抛 CancelledException。

PdfParser

```
class PdfParser implements DocumentParser {
    PdfParser({required this.textExtractor});
}

final PdfParser parser = PdfParser(
    textExtractor: TikaCliTextExtractor(
        processRunner: const IoProcessRunner(),
        tikaHome: r'tools\tika-app',
    ),
);
final ParseResult result = await parser.parse(r'datasets\samples\sample.pdf');
// result.document?.attributes['pageCount'] 与页列表长度一致
```

页文本来自 textExtractor。每页一个 DocumentPage, attributes['pageCount'] 与页列表长度一致。空文件和目录是 failed, 不会去启动提取器。ProcessStartException 变成 dependencyUnavailable (缺 Java) 。ProcessTimeoutException 变成 timeout。TextExtractionException 变成 corruptedInput, stderr 在 detail。取消抛 CancelledException。

成功时 attributes['stageMs'] 是各段毫秒数, 值为 int。stat 从 File.stat 到空文件判断结束。assemble 是拼出 ParsedDocument。提取器写入的 process_start、process_wait、stream_drain、xhtml_split、text_fallback 会保留。ParseResult.duration 仍是整次解析的墙钟, 不改成这几段之和。失败结果没有 stageMs。

ImageParser

```
class ImageParser implements DocumentParser {
    const ImageParser();
}

final ParseResult result = await const ImageParser().parse(
    r'datasets\samples\sample.png',
);
// result.document?.fullText 为空
// result.document?.imageAsset?.format == 'png'
```

读 JPEG 和 PNG 的格式、宽高、色彩模式。正文为空。没有 EXIF 不报错，orientation 为 null。空文件、伪装扩展名、解码失败都是 corruptedInput。缺失文件是 failed。取消抛 CancellationException。

TextExtractor / ExtractedText

```
abstract interface class TextExtractor {
    String get name;
    String get version;
    Future<ExtractedText> extract(String absolutePath, {CancellationToken? cancel});
}

class ExtractedText {
    const ExtractedText({
        required this.pages,
        required this.extractorName,
        this.title,
        this.attributes = const <String, Object?>{},
    });
    String get fullText;
    bool get hasText;
}

const ExtractedText text = ExtractedText(
    pages: <DocumentPage>[
        DocumentPage(pageNumber: 1, text: '第一页'),
        DocumentPage(pageNumber: 2, text: '第二页'),
    ],
    extractorName: 'tika-cli',
);
// text.fullText == '第一页\n\n第二页'
```

fullText 用空行连接各页。文档问题由实现抛出调用方已经声明的异常，再由 PdfParser 收成 ParseFailure。取消抛 CancellationException。

TikaCliTextExtractor / TextExtractionException / pageTextsFromXhtml

```
class TikaCliTextExtractor implements TextExtractor {
    TikaCliTextExtractor({
        required this.processRunner,
        required this.tikaHome,
        this.javaExecutable = 'java',
        this.version = '4.0.0',
        this.timeout = const Duration(seconds: 60),
        String? classpathSeparator,
```

```

    });
}

class TextExtractionException implements Exception {
    const TextExtractionException(this.message, {this.exitCode, this.stderr});
}

List<String> pageTextsFromXhtml(String xhtml);

final List<String> pages = pageTextsFromXhtml(
    '<div class="page">第一页</div><div class="page">第二页</div>',
);
// pages.length == 2
// pageTextsFromXhtml('<html></html>') 返回空列表

```

先跑 TikaCLI -x。XHTML 里有 class="page" 的 div 就按页切开；没有则再跑 -t，整篇作为一页。正文只取 stdout。classpath 在 Windows 上用；，其他系统用：。-x 起不来或超时会直接抛，不再跑 -t；-x 有退出码但没有 page div 时会回退 -t。两种模式都拿不到正文时抛 TextExtractionException。Java 起不来由 ProcessRunner 抛 ProcessStartException。pageTextsFromXhtml 没有 page div 时返回空列表，不抛异常。HTML 实体支持 amp lt gt quot apos nbsp 以及十进制、十六进制数字实体。

attributes['stageMs'] 记录这次抽取的毫秒数。process_start、process_wait、stream_drain 来自 -x 那次进程。process_wait 含 JVM 启动、扫描 classpath 和 Tika 本体，三者没有再拆开。xhtml_split 是 pageTextsFromXhtml。text_fallback 在已经分出页时为 0；走到 -t 时是第二次进程的整段 duration。

5. 进程、执行与取消

ProcessRunner / IoProcessRunner / ProcessRunResult

```

abstract interface class ProcessRunner {
    Future<ProcessRunResult> run(
        String executable,
        List<String> arguments, {
        String? workingDirectory,
        Map<String, String>? environment,
        Duration? timeout,
        CancellationToken? cancel,
    });
}

class IoProcessRunner implements ProcessRunner {
    const IoProcessRunner();
}

class ProcessRunResult {
    const ProcessRunResult({
        required this.exitCode,
        required this.stdout,
        required this.stderr,
        this.duration = Duration.zero,
        this.processStart = Duration.zero,
        this.processWait = Duration.zero,
        this.streamDrain = Duration.zero,
    });
    bool get isSuccess;
}

```

```

    Map<String, Object?> toJson();
}

const ProcessRunResult run = ProcessRunResult(
  exitCode: 0,
  stdout: '<div class="page">ok</div>',
  stderr: '',
);
// run.isSuccess == true
// run.toJson() 只记 stdout、stderr 的长度
// 真 Java 用 const IoProcessRunner(), 只出现在 integration 用例里

```

这是包内唯一允许启动进程的类型。IoProcessRunner 调用 Process.start。起不来抛 ProcessStartException。超时先杀掉进程再抛 ProcessTimeoutException。取消杀掉进程并抛 CancelledException。toJson 只记 stdout/stderr 的长度，不记内容，并带上 durationMs、processStartMs、processWaitMs、streamDrainMs。duration 是整次调用的墙钟。processStart 到 Process.start 返回为止，不含 JVM 初始化。processWait 从进程已启动到退出码返回，Java 这一段含 JVM 启动和程序本体。streamDrain 是退出后把 stdout、stderr 收完。三段之和可以小于 duration，中间差在挂接流监听。超时和取消不返回 ProcessRunResult。默认单测注入假实现；真 Java 只放在 integration tag 的用例里，用 dart test --tags integration 跑。

ProcessStartException / ProcessTimeoutException

```

class ProcessStartException implements Exception {
  const ProcessStartException(this.executable, this.message);
}
class ProcessTimeoutException implements Exception {
  const ProcessTimeoutException(this.executable, this.timeout);
}

const ProcessStartException missing = ProcessStartException('java', 'not found');
const ProcessTimeoutException timedOut = ProcessTimeoutException(
  'java',
  Duration(seconds: 60),
);
// PdfParser 把 missing 收成 dependencyUnavailable, 把 timedOut 收成 timeout

```

两者都是异常，不是 ParseFailure。PdfParser 负责转换。

Executor / ImmediateExecutor

```

abstract interface class Executor {
  Future<T> run<T>(Future<T> Function() action);
}
class ImmediateExecutor implements Executor {
  const ImmediateExecutor();
}

const Executor executor = ImmediateExecutor();
final String value = await executor.run(() async => 'ok');
// value == 'ok', 在当前 isolate 上执行

```

run 执行 action 并返回它的结果。默认实现就在当前 isolate 上直接调用，异常原样抛出。不抛自己的异常。

CancellationToken / CancellationTokenSource / CancelledException

```
class CancelledException implements Exception {
  const CancelledException([this.message = 'Operation cancelled']);
}

class CancellationToken {
  bool get isCancelled;
  void throwIfCancelled();
  Future<void> get whenCancelled;
  void addListener(void Function() listener);
  void removeListener(void Function() listener);
}

class CancellationTokenSource {
  CancellationToken get token;
  bool get isCancelled;
  void cancel();
}

final CancellationTokenSource source = CancellationTokenSource();
source.cancel();
// source.token.isCancelled == true
// source.token.throwIfCancelled() 抛 CancelledException
```

令牌没有公开构造函数，只能从 `source` 拿。`cancel` 可以多次调用。已经取消后再 `addListener` 会立刻调用监听器。`throwIfCancelled` 抛不带自定义消息的 `CancelledException`。`whenCancelled` 在已经取消时返回一个已经完成的 `future`。监听器应短小，不要在里面再取消同一条流水线的锁。仅限当前 `isolate`。

6. 公开符号清单

对照 `lib/core_parsing.dart` 的导出文件：

`defaultIngestExtensions`、`FileScanner`、`IngestionService`、`MetadataExtractor`、`FileMetadataExtractor`、`mimeTypeForExtension`、`DocumentPage`、`FileMetadata`、`ImageAsset`、`IngestRecord`、`IngestionReport`、`IngestProgress`、`ParseOutcome`、`ParseFailureKind`、`ParseFailure`、`ParsedDocument`、`ParseResult`、`TextExtractionException`、`TikaCliTextExtractor`、`pageTextsFromXhtml`、`DocumentParser`、`ParserRegistry`、`StandardParserRegistry`、`normalizeParserExtension`、`DocxParser`、`ImageParser`、`PdfParser`、`TxtParser`、`CancelledException`、`CancellationToken`、`CancellationTokenSource`、`Executor`、`ImmediateExecutor`、`IoProcessRunner`、`ProcessRunResult`、`ProcessStartException`、`ProcessTimeoutException`、`ProcessRunner`、`ExtractedText`、`TextExtractor`。